BIG-GEM

The Big-GEM Theory

Heterogeneous GPU-Accelerated RTL Simulation

Native DSP48E2, CARRY4, and SRLC32E execution in NVIDIA GEM

Technical Project Report

Takneek PS – Zenith

Deliverable E

Repository: `https://github.com/Pratham-015/GEM`

Measured revision: `23fc43f`

Report date: 2 September 2026

Project: Big-GEM / The Big-GEM Theory
Base system: NVIDIA GEM GPU-accelerated RTL simulator
Extension: Heterogeneous native execution of DSP48E2, CARRY4, and SRLC32E
Primary measured platform: NVIDIA GeForce RTX 4050 Laptop GPU (SM 8.9)
Author recorded in the measured revision: Pratham Sharma

Contents

1. Project Overview

1.1 Abstract

Big-GEM extends NVIDIA GEM from a homogeneous one-bit And-Inverter Graph (AIG) simulator into a heterogeneous simulator that preserves and executes three FPGA primitives: DSP48E2, CARRY4, and SRLC32E. Yosys 0.68 and its Slang frontend elaborate SystemVerilog while black-box interception and a DSP-specific techmap retain macro identity. The host constructs explicit named-port macro records, classifies same-cycle and clock-boundary dependencies, fuses legal CARRY4 chains, and creates warp-padded 64-bit structure-of-arrays buffers. A custom cooperative CUDA kernel interleaves original Boomerang Boolean propagation with typed macro gather, shared-memory staging, native 64-bit evaluation, output publication, and a single synchronized state-commit boundary.

Correctness was tested through the production RTL-to-CUDA path and against independent Python, literal SystemVerilog, and installed Xilinx UNISIM models. CARRY4 covered its complete 1,024-input truth table plus randomized cases; GPU differential totals were 3,024 CARRY4 slices, 2,009 fused 60-bit chains, 2,336 DSP cycles, and 2,100 SRLC32E cycles. Four randomized production DAG topologies and the exact DSP→CARRY4→SRLC32E→DSP chain matched independent RTL with zero reported mismatches.

Production throughput was measured with 12 warm-ups and seven retained launches. The only identical-workload comparison supported by both official upstream and the modified simulator is Boolean-only: 59,232 versus 59,322 simulated cycles/s, or 1.0015× (+0.153%). A separate 9-partition heterogeneous stress design scaled from 5,273 cycles/s at one block to 16,677 cycles/s at 12 blocks (3.163×). No valid heterogeneous upstream speedup is claimed: the historical shredded-netlist comparison failed its RTL correctness gate.

1.2 Problem and Contribution

The problem statement identifies a structural inefficiency in conventional GEM: Yosys decomposition converts wide arithmetic and stateful primitives into large one-bit networks. Big-GEM changes the representation and execution path rather than adding an isolated macro microbenchmark. Its concrete contributions are:

1. a Yosys 0.68 macro-preserving and DSP-normalizing frontend;
2. a heterogeneous host graph with stable port identities, dependency timing, and explicit combinational levels;
3. separate bit-packed Boolean storage and 64-bit aligned, warp-padded macro state/I/O storage;
4. a custom CUDA schedule that places macro operations between Boolean settling barriers and commits DSP/SRL state at one global edge; and
5. reproducible differential, throughput, block-scaling, and Nsight Compute evidence.

Evidence boundary. Results in this report were regenerated from the recorded project revision. Benchmark provenance should be kept synchronized with the final submission.

2. Requirements and Constraints

Table 1: Problem-statement requirements and demonstrated status.

Requirement	Status	Evidence in the current repository
Preserve DSP48E2, CARRY4, SRLC32E through Yosys	**Verified**	Production JSON checks in `verif/full_integration_test.py`; Yosys 0.68 observed.
Mixed 1-bit and 48-bit heterogeneous scheduling	**Verified**	Typed dependency graph and production exact-chain/random-DAG differentials.
64-bit aligned contiguous macro buffers	**Verified**	`src/macro_layout.rs`; four layout tests, including warp-scale non-overlap.
Native GPU ALU macro evaluation	**Verified**	`csrc/gem_macros.cuh` executed on GPU and differentially checked.
Correct macro-to-macro sequencing	**Verified**	Explicit levels; fused CARRY4 chains; exact-chain and randomized production tests.
Shared memory and warp synchronization	**Verified**	12,288-byte macro tile, `__ballot_sync`, block barriers, and cooperative grid barriers.
Yosys 0.68 and IEEE 1800-2012 syntax	**Verified**	Installed Yosys 0.68; Slang parses the SystemVerilog-2012 feature test.
Single global clock and zero initialization	**Verified**	Multiple macro clocks are rejected; state buffers are zero-created and checked.
Throughput versus unmodified GEM	**Partial**	Identical Boolean workload measured; no valid equivalent heterogeneous upstream comparison.
Nsight bandwidth/divergence report	**Verified**	Four current production-kernel profiles are stored as raw CSV and parsed JSON.

The required DSP configuration is enforced structurally: AREG, BREG, CREG, DREG, ADREG, and MREG are zero and PREG is one. Unsupported parameterizations cause the DSP techmap to fail instead of being silently simulated with different latency. The simulator assumes one global clock domain and initializes all internal macro state to zero; INIT-string parsing remains intentionally out of scope, exactly as permitted by the problem statement.

3. System Architecture

3.1 End-to-End Data Path

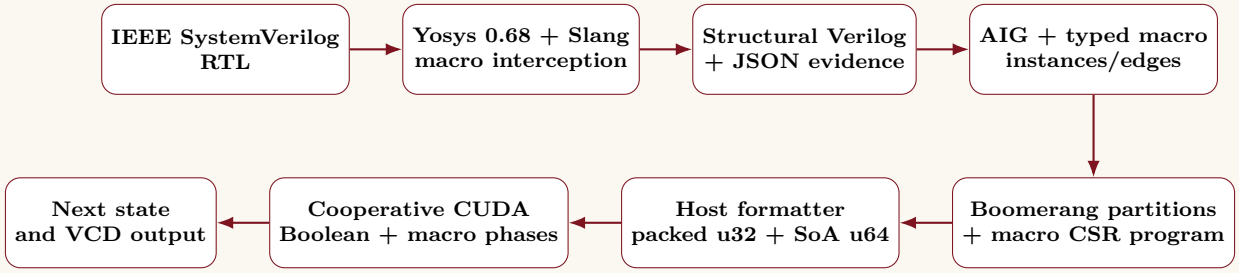

Figure 1: Actual Big-GEM production path. No standalone macro microkernel is substituted for the measured simulator.

3.2 Frontend and Macro Preservation

`scripts/synthesize_macros.py` checks the installed version string, invokes `read_slang` with an IEEE 1800-2017 mode that accepts the required 2012 syntax, elaborates the requested top, and runs synthesis/ABC around black-box macro declarations. `aigpdk/dsp48e2_ps_map.v` transforms a supported raw DSP48E2 into `GEM_DSP48E2` with two state bits and one pre-adder-control bit. CARRY4 and SRLC32E remain named black boxes. JSON is not the runtime parser input; it is retained as machine-checkable preservation evidence. The runtime host parser consumes the emitted structural Verilog and constructs `NetlistDB`, AIG, and macro records.

The DSP control mapping converts supported OPMODE and INMODE settings into the simplified control state used by the GPU evaluator. Plain-add ALUMODE and supported INMODE encodings are recognized by the map. Static pipeline parameters outside the required subset are rejected.

4. Heterogeneous Graph and Scheduling Equations

4.1 Typed Graph

Let the execution graph be

$$G = (V, E), \qquad V = V_B \cup V_C \cup V_S \cup V_D,$$

where V_B contains one-bit AIG operations, V_C fused CARRY4-chain nodes, V_S SRLC32E nodes, and V_D DSP48E2 nodes. The sets are disjoint and each macro instance retains named input/output ports and bit indices. Define

$$T(v) \in \{\text{AIG}, \text{CARRY}, \text{SRL}, \text{DSP}\}.$$

An edge records producer cell/port/bit and consumer cell/port/bit. The frontend traces through intervening AIG logic, so a macro dependency is recovered even when ordinary Boolean nodes lie between producer and consumer.

Each macro input has role $r_i \in \{\text{comb}, \text{next}\}$ and each macro output has role $r_o \in \{\text{comb}, \text{reg}\}$. The implemented timing classifier is exactly

$$\tau(e) = \begin{cases} \text{same}, & r_i(e) = \text{comb} \ \wedge \ r_o(e) = \text{comb}, \\ \text{edge}, & \text{otherwise}. \end{cases}$$

CARRY O/CO and SRL Q/Q31 are combinational outputs; DSP P is registered. CARRY inputs and SRL address are combinational; DSP operands/control and SRL D/CE are next-state inputs.

4.2 Combinational Levelization

Only nodes with combinational outputs participate in the same-cycle macro DAG $G_c = (V_c, E_c)$, where $E_c = \{e \in E : \tau(e) = \text{same}\}$. The host uses Kahn levelization. Its equivalent recurrence is

$$\ell(v) = \begin{cases} 0, & \text{pred}_{E_c}(v) = \varnothing, \\ 1 + \max\limits_{u \in \text{pred}_{E_c}(v)} \ell(u), & \text{otherwise}. \end{cases}$$

The cohort at level k is $M_k = \{v \in V_c : \ell(v) = k\}$. A nonempty residual indegree after Kahn traversal is rejected as a combinational macro cycle. DSP nodes do not enter G_c because PREG=1 makes P a cycle-boundary source.

Legal adjacent CARRY4 instances are fused before levelization when CO[3] drives the next CIN, CYINIT is inactive, and total width is at most 60 bits. For a fused chain, the internal macro-to-macro edges disappear and one native operation publishes all O/CO bits. Non-fused same-cycle macro edges are ordered by ℓ and communicated through explicitly allocated output-bit slots in the packed Boolean-state row. Thus ordering is explicit, although the current implementation does not forward a producer's 64-bit register value directly to a consumer register.

4.3 Partition and Block Assignment

The host assigns endpoint cones to partitions using a deterministic load-balancing rule. Partition validity is checked during construction, and valid partitions are then mapped to CUDA blocks using a second load-balancing step. This keeps the Boolean work distributed across available blocks without changing the macro dependency order.

4.4 Per-Cycle Heterogeneous Schedule

Each simulated cycle follows a simple two-stage process. First, the current registered macro outputs are exposed and the Boolean network is settled while macro levels are executed in topological order. The simulator then gathers the next-state inputs for DSP48E2 and SRLC32E2-style state, commits the synchronous state at the single global rising edge, and performs the final Boolean propagation needed to expose the newly registered outputs.

Every macro gather, evaluation, and publication stage is separated by the required cooperative synchronization boundaries. This preserves dependency ordering while allowing independent macro instances to execute concurrently.

5. GPU Memory Hierarchy and Data Movement

5.1 Layout Equations

Boolean state remains bit-packed in 32-bit words. Macro state and I/O occupy separate CUDA allocations of 64-bit words. For each macro kind, instances are padded to a whole warp. The padded stride is

$$s_t = 32 \left\lceil \frac{n_t}{32} \right\rceil .$$

Each field then stores one 64-bit value per instance, so adjacent lanes access adjacent words. The allocator uses eight I/O fields for CARRY and DSP and four for SRL. DSP P and the SRL's 32-bit state each consume one warp-padded 64-bit state slot; CARRY is stateless. CUDA allocations and every field address are therefore 8-byte aligned, and adjacent lanes access adjacent 64-bit words for one field.

5.2 FPGA Primitive to GPU Hierarchy

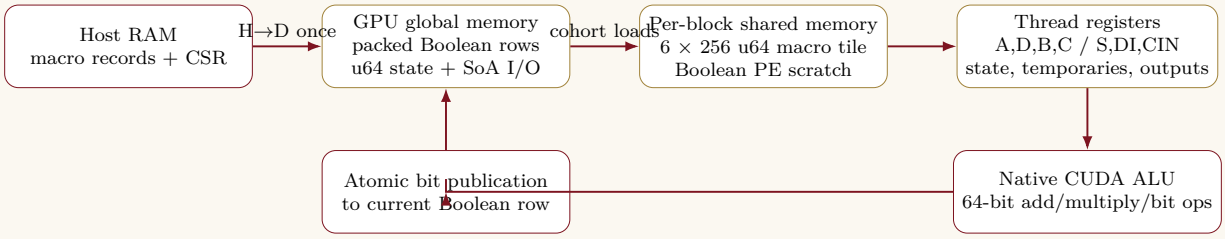

Figure 2: Actual macro data movement. Global memory is the grid-wide exchange boundary; shared memory is a block-local staging tile; arithmetic temporaries remain in registers.

Table 2: Primitive-specific mapping onto the measured CUDA hierarchy.

Primitive	Datapath	State	Global memory	Shared tile	Registers / execution
DSP48E2	27+27, 18, 45, 48 bit	48-bit P	Five SoA input fields, P state, output bit slots	Five inputs + one state word/thread	Signed 64-bit pre-add, multiply, and masked ALU select
CARRY chain	4–60 bit	None	Three input fields and O/CO output slots	S, DI, CIN words/thread	One 64-bit add plus Boolean transforms
SRLC32E	5-bit address, 1-bit I/O	32-bit SRL	One packed control field, state word, Q/Q31 slots	Control + state word/thread	Shift/mask and dynamic register read

The full measured kernel uses 16,640 bytes of static shared memory per block: 12,288 bytes are the dedicated macro tile; the remainder is Boolean metadata, writeout, state, and script-index storage. Nsight reports 124 registers/thread. The kernel uses `__ballot_sync` to form selected macro

cohorts and `__all_sync` to assert type uniformity. Block-local phases use `__syncthreads`; grid-wide visibility uses `cooperative_groups::this_grid().sync()`.

6. Native Hardware Macro Semantics

6.1 DSP48E2 Simplified Subset

All DSP operands use signed two's-complement arithmetic at the widths defined by the problem statement. The pre-adder either forms $A + D$ or passes A directly, and the multiplier produces the required 45-bit product.

At the rising clock edge, the 48-bit P register is updated according to the simplified control state:

- State 0: $P_{\text{next}} = C$
- State 1: $P_{\text{next}} = M$
- State 2: $P_{\text{next}} = P_{\text{current}} + M$

The implementation preserves the specified signed widths and 48-bit result behavior.

6.2 CARRY4 and Fused Chains

The literal specification is

$$C_0 = \text{CYINIT} \vee \text{CIN}, \quad C_{i+1} = (S_i \wedge C_i) \vee (\neg S_i \wedge DI_i),$$

$$O_i = S_i \oplus C_i, \qquad CO_i = C_{i+1}.$$

For a fused CARRY4 chain, the implementation evaluates the chain as one native operation while producing the same O and CO bits as the individual CARRY4 equations. The implementation supports the tested chain-width limit and safely handles longer chains through segmentation.

6.3 SRLC32E

SRLC32E keeps a 32-bit internal state. On a rising edge with CE asserted, the state shifts toward higher indices and D is loaded into index 0; otherwise the state is unchanged. Q continuously reads the selected address and Q31 reads bit 31. The persistent state is initialized to zero, and DSP and SRL next-state inputs are gathered before either state array is updated.

7. Verification and Validation

Table 3: Executed verification evidence on 2 September 2026.

Test	Result	Coverage
Rust workspace, all targets, CUDA feature	PASS	1 partitioner unit test; 4 memory tests; 4 parser/DAG integration tests; all binaries compiled.
CARRY4 differential	PASS	3,024 slices, including complete 1,024-case truth table; 2,009 60-bit chains.
DSP48E2 differential	PASS	2,336 sequential vectors with signed/boundary/random cases.
SRLC32E differential	PASS	2,100 sequential vectors with CE/address changes.
Independent references	PASS	Python, literal SystemVerilog/Icarus, installed Xilinx UNISIM, and production CUDA.
Integrated RTL pipeline	PASS	RTL→Yosys→JSON→partition→CUDA→VCD.
Exact required chain	PASS	DSP→CARRY4→SRLC32E→DSP on 1 and 4 blocks; 2,709 values checked/run, zero mismatches.
Random production DAGs	PASS	4 seeded topologies, 192 stimulus cycles, 1,029 values/run, zero mismatches.
Wide multi-partition path	PASS	9 partitions, 16 blocks, 18 complete 8,192-bit result frames, zero mismatches.
Nsight production profiles	PASS	Exactly one production simulation kernel selected per profile.
Compute Sanitizer	BLOCKED	WDDM debugger initialization failed and this device/driver combination was reported unsupported.

The production simulator's optional `-check-with-cpu` path provides a second schedule/state sanity check. Expected external outputs are nevertheless generated independently by Verilog/UNISIM rather than by reusing the CUDA evaluator. The full regression command completed successfully:

```
python3 verif/full_integration_test.py
```

8. Benchmarking and Numerical Analysis

8.1 Metric and Timing Boundary

The primary metric is

$$\mathrm{CPS} = \frac{N_{\text{simulated cycles}}}{t_{\text{launch+kernel+sync}}}.$$

Each workload is synthesized and partitioned once. The measured interval wraps one production cooperative-kernel launch and mandatory device synchronization; it excludes parsing, allocation, H→D/D→H transfer, synthesis, partitioning, and output generation. Twelve launches are discarded before seven measurements. State and deterministic inputs are reset outside each measured interval. The table uses the median CPS; "runtime" is calculated as cycles/median CPS, while CV is sample standard deviation divided by mean CPS.

8.2 Production Workload Throughput

Table 4: Current production-kernel throughput (seven retained samples).

Workload	Cycles	Blocks	AIG	Macros D/C/S	Runtime (ms)	Median CPS
Boolean heavy	2,000	4	507	0/0/0	25.445	78,600
DSP heavy	2,000	4	4,464	32/0/0	95.050	21,041
Carry heavy	1,000	4	2,880	0/128/0	636.486	1,571
SRL heavy	2,000	4	576	0/0/128	46.847	42,692
Heterogeneous small	1,000	4	1,120	4/8/8	129.535	7,720
Heterogeneous mixed	1,000	4	3,520	16/32/32	281.028	3,558
Deep dependency	200	4	1,981	0/64/0	428.885	466
Heterogeneous large	200	4	8,704	32/128/128	190.018	1,053
Occupancy stress	1,000	16	82,385	1/1/1	60.004	16,666

The data show workload cost, not a cross-workload speedup. Carry-heavy and deep dependency cases are slow because every combinational macro level forces another complete Boolean settling pass and grid barriers. SRL-heavy is much faster than carry-heavy despite equal macro count because independent SRLs share a shallow level. Mixed scaling drops from 7,720 CPS (small) through 3,558 CPS (medium) to 1,053 CPS (large) as AIG and macro work increase. These values demonstrate functional scaling but do not isolate the benefit of preserving a macro.

All seven-sample coefficients of variation were below 0.43%; mixed and large were 0.015% and 0.030%, respectively. This repeatability does not remove systematic issues such as laptop power state or the dirty-tree provenance.

8.3 Equivalent Upstream Comparison

Table 5: Identical macro-free Boolean workload on official upstream and modified GEM.

Implementation	Cycles	Mean runtime (ms)	Median CPS	CPS CV
Official upstream `9e913f9`	2,000	33.779	59,231.5	0.361%
Modified Big-GEM `23fc43f`	2,000	33.712	59,322.3	0.132%

Measured and calculated values are separated as follows:

$$\text{speedup} = \frac{59{,}322.260}{59{,}231.513} = 1.001532\times, \qquad \text{gain} = 0.1532\%.$$

This is parity within a small fraction of one percent, not evidence of a macro speedup. It does show that the Boolean-only fast path preserves upstream throughput on this workload. The benchmark runner attempted a same-RTL shredded-upstream versus preserved-modified experiment, but at least one representation failed the independent RTL differential. Its performance number was therefore suppressed. No heterogeneous baseline speedup is reported.

8.4 Useful Multi-Partition Scaling

Table 6: Nine-partition occupancy-stress block sweep, 1,000 cycles, seven samples.

Blocks	1	4	8	9	12	16	20	28/40
CPS	5,273	10,776	13,056	16,670	16,677	16,665	16,661	11,452/11,451

The best retained median occurs at 12 blocks:

$$\frac{16{,}676.956}{5{,}272.581} = 3.16296\times \quad \text{or} \quad 216.30\% \text{ gain over one block.}$$

Only nine blocks have useful partition work. Additional cooperative blocks are idle but still participate in grid synchronization; performance is flat through 20 blocks and falls by about 31% at 28–40 blocks. This is direct evidence that partitioning exposes useful parallel work, and equally direct evidence that oversizing the cooperative grid is harmful.

8.5 Nsight Compute Results

Table 7: Current production-kernel Nsight Compute counters.

Workload	Grid	Ach./theor. occ.	Divergent targets	Predicated lanes	DRAM MB/s	LD/ST sectors
Exact chain	1	16.67/33.33%	3.764%	70.09%	6.52	6.63/2.89
Mixed	4	16.67/33.33%	1.212%	75.16%	1.95	7.36/3.73
Large	4	16.67/33.33%	1.078%	75.59%	2.73	7.52/3.70
Occupancy stress	16	16.71/33.33%	1.306%	83.22%	15.19	9.72/4.06

The profiled symbol was `simulate_v1_noninteractive_simple_scan`; each profile selected exactly one production kernel. The environment was Nsight Compute 2025.2.1.0, CUDA 12.9, driver 596.49,

and RTX 4050 Laptop GPU. DRAM peak utilization rounded to 0.00–0.01%, so these tests are not bandwidth-saturated. The principal limits are synchronization, insufficient useful blocks in three workloads, and a 124-register/thread kernel whose theoretical occupancy is 33.33%. Branch-target divergence is low (1.08–3.76%), but predicated-lane utilization of 70–83% shows that branch uniformity is not equivalent to full lane efficiency.

The load/store sector ratios are transaction-density counters, not a direct coalescing-efficiency percentage because the kernel mixes access widths and instruction sites. They therefore support neither a claim of perfect coalescing nor a bandwidth bottleneck. The SoA address mapping is structurally contiguous; instruction-level coalescing attribution remains future profiling work.

9. Reproducibility

Table 8: Measured software and hardware configuration.

Item	Recorded value
Repository	`https://github.com/Pratham-015/GEM`
Git revision	`23fc43f`; dirty working tree
GPU / compute capability	NVIDIA GeForce RTX 4050 Laptop GPU / 8.9
Driver / VRAM	596.49 / 6,141 MiB
CUDA compiler	CUDA 12.9, NVCC 12.9.86
Nsight Compute	2025.2.1.0
Yosys	0.68, git 38e001a6f, Slang frontend
Rust	rustc 1.98.0
CUDA launch	256 threads/block; cooperative production kernel

9.1 Commands

```
python3 verif/full_integration_test.py
python3 benchmark/run_benchmarks.py --repetitions 7
python3 benchmark/profile_boomerang_ncu.py
python3 benchmark/profile_boomerang_ncu.py \
  --workload mixed_heterogeneous --skip-prepare
python3 benchmark/profile_boomerang_ncu.py \
  --workload large_scale --skip-prepare
python3 benchmark/profile_boomerang_ncu.py \
  --workload occupancy_stress --blocks 16 --skip-prepare
python3 benchmark/profile_block_sweep.py \
  --workload occupancy_stress --repetitions 7
```

Machine-readable results are in `benchmark/results/latest.json`, `benchmark/results/block_sweep_occupancy_stress.json`, and `benchmark/nsight_*.json`; raw Nsight CSV files are stored beside the JSON.

10. Risks, Limitations, and Failure Modes

Table 9: Known limitations at report generation time.

Limitation	Consequence / mitigation	Status
No valid heterogeneous upstream differential	Native-preservation speedup cannot be claimed; repair the shredded golden flow and rerun equivalent RTL.	Open
Dirty-tree measurements	Commit hash alone cannot reconstruct every measured byte; commit changes and rerun archival evidence.	Open
Two full Boolean walks per macro phase	Deep combinational macro levels incur repeated AIG work and grid barriers.	Measured
Macro exchange uses global bit slots	Correct across blocks, but direct word-level macro forwarding is not implemented for non-fused edges.	Architectural limit
124 registers/thread	Theoretical occupancy is 33.33%; small grids achieve about 16.67%.	Measured
Compute Sanitizer unavailable	Memory/race diagnostics were not obtained from this WDDM/driver/device combination.	Blocked
SystemVerilog breadth	Valid hidden designs beyond the exercised hierarchy/generate/signed tests may expose parser assumptions.	Residual risk

No result supports the phrases “zero divergence”, “fully coalesced”, or “macro acceleration versus upstream”. The report intentionally does not make those claims.

11. Results Summary and PS Coverage

Table 10: Deliverable E coverage.

Deliverable E requirement	Coverage	Location
Mathematical scheduling definition	Complete	Section 4: typed graph, edge timing, level recurrence, partition/block assignment, and two-phase cycle equation.
FPGA-to-GPU architecture diagrams	Complete	Figures 1–2 and primitive memory-mapping table.
CUDA architecture and memory layout	Complete	Sections 3–6, including global/shared/register placement and synchronization.
Extensive numerical analysis	Complete with limitation	Section 8: nine workloads, upstream parity, block scaling, Nsight, uncertainty, and missing external data.
Benchmarking under a unique header	Complete	Section 8 is titled “Benchmarking and Numerical Analysis”.
Reproducibility	Complete	Section 9 records versions, revision, commands, and evidence paths.

12. Conclusion

The current repository contains a real integrated heterogeneous execution path: SystemVerilog reaches Yosys 0.68, named macros survive synthesis, the host builds typed instances and dependencies, the allocator creates aligned SoA storage, and the production cooperative CUDA kernel executes Boolean and native macro phases with explicit synchronization and clock-state semantics. The strongest evidence is correctness: exhaustive/differential primitive tests, independent UNISIM and RTL references, exact required-chain simulation, randomized production DAGs, and a wide multi-partition simulation all pass.

Performance evidence is more limited. The extension preserves Boolean-only throughput within 0.153% of upstream and exposes a 3.163× useful scaling gain on a 9-partition stress graph. Nsight demonstrates low branch-target divergence but low absolute bandwidth and occupancy constrained by workload size and register use. Because the heterogeneous shredded baseline failed correctness, the submission does not yet demonstrate a valid native-macro speedup over upstream GEM. This is the most important performance-evidence gap before judging.

13. References and Project Sources

1. Takneek PS – Zenith, *The Big-GEM Theory*, supplied six-page problem statement, 2026.
2. Z. Guo, Y. Zhang, R. Wang, Y. Lin, and H. Ren, "GEM: GPU-Accelerated Emulator-Inspired RTL Simulation," DAC 2025.
3. NVIDIA, official GEM repository, `https://github.com/NVlabs/GEM`, upstream comparison commit `9e913f9`.
4. Big-GEM implementation sources: `src/aig.rs`, `src/flatten.rs`, `src/macro_layout.rs`, `csrc/kernel_v1_impl.cuh`, and `csrc/gem_macros.cuh`.
5. Verification and measurement sources: `verif/full_integration_test.py`, `verif/host/diff_harness.py`, and `benchmark/run_deliverable_d.py`.

A. Appendix A – Exact Evidence Files

Evidence	Purpose
`benchmark/results/latest.json`	Raw commands, environment, seven samples/workload, summaries, and upstream comparison.
`benchmark/nsight_boomerang.csv/json`	Exact-chain production-kernel counters.
`benchmark/nsight_mixed_heterogeneous.csv/json`	Mixed production-kernel counters.
`benchmark/nsight_large_scale.csv/json`	Large production-kernel counters.
`benchmark/nsight_occupancy_stress.csv/json`	Multi-partition stress production-kernel counters.
`benchmark/results/block_sweep_occupancy_stress.json`	Per-block-count raw stdout and seven throughput samples.
`verif/host/diff_harness.py`	Independent CPU/RTL/UNISIM/GPU primitive differential harness.
`verif/full_integration_test.py`	Full synthesis, parser, scheduler, CUDA, VCD, randomized, and Nsight regression.

B. Appendix B – Final Technical Consistency Matrix

Report claim	Source implementation	Executed evidence
Macro preservation	`scripts/synthesize_macros.py`, `aigpdk/dsp48e2_ps_map.v`	JSON cell-count assertions
Same-cycle macro levels	`src/aig.rs`	exact-chain/random-DAG VCD differentials
64-bit SoA storage	`src/macro_layout.rs`	`tests/macro_memory_layout_test.rs`
Shared macro tile	`csrc/kernel_v1_impl.cuh`	cuobjdump resource assertion; Nsight 16,640 B/block
Global edge semantics	gather-before-commit CUDA schedule	DSP/SRL multi-cycle and integrated RTL tests
Throughput numbers	`benchmark/run_deliverable_d.py`	`benchmark/results/latest.json`
Warp/memory counters	`benchmark/profile_boomerang_ncu.py`	raw Nsight CSV + parsed JSON